Chapter one

Introduction to Network Administration

Networking Protocols and Architecture

What is communication Protocols?

Communication Protocol: - define the manner in which peer processes communicate b/n computer hardware devices. The protocols give the rule for such things as the passing of messages, the exact formats of the message and how to handle error conditions.

If two computers are communicating and they both follow the protocol(s) properly, the exchange is successful, regardless of what types of the machines they are and what operating systems are running on the machines. As long as the machines have software that can manage the protocol, communication is possible.

Essentially, therefore, a communication protocol is a set of rules that coordinates the exchange of information.

What is Host?

A host is typically refers to a computer that provide information or communication service.

What are a Gateway, a Router and Routing?

A gateway or Router: - is a computer that interconnects two or more networks and passes packets from one to another.

The process by which the paths that packets travel across the network or inter-network are chosen is known as **routing**.

Protocol Layering

A wide range of problems may arise in packet-based data communication. These include the following:

- ☞ Host failure: A host or gateway may fail due to a hardware or software crash.
- ☞ **Link failure**: A transmission link may be damaged or disconnected.
- ☞ **Network congestion**: networks have a finite capacity which cannot be exceeded.
- ☞ **Packet delay or loss**: Packets are sometimes lost during transmission or may experience excessive delay.
- ☞ **Data corruption**: Transmission error may corrupt the data being transmitted.
- ☞ **Data duplication or packets out-of-sequence**: Where more than one router exists in network connection, it is possible for transmitted packets to arrive out of sequence.

Layer Services

Connection-Oriented and Connectionless Services

- In connection oriented service the sender of the data first establishes a logical connection with the receiver of the data, use the connection (send the data) and then terminate the connection. During the establishment of the connection, a fixed route that all packets will take is defined, and information necessary to match packets to their session and defined rout is stored in the memory tables in the gateways.
  Connection-oriented protocols provide in-sequence delivery; that is, the service user receives packets in the order.
- In connection-less (Datagram) service there is no initial end-to-end setup for a session; each packet is independently routed to its destination. When a packet is ready, the host computer sends it in to the gateway. The gateway examines the destination address of the packet and passes the packet along to another gateway, chosen by the route-find algorithm.

Reliable and Unreliable Services

Services can also be classified according to the 'Quality of Service' that they provide to the layer above. There are two types of service quality: Reliable and Unreliable

- **A Reliable Service**: - is one that endeavors never to lose data during a transfer and provide error-free data to the service user.
    - In such a scheme the receiver is required to acknowledge the receipt of each item of data, to ensure that no data is lost in transmission.
    - In addition to this, the receiver checks each data item received for errors, informing the source if an error is detected and that another copy of the affected data should be sent.
- The acknowledgement process required for reliable service introduces **delay** and **overhead**. There are some cases when it is more important for the service to be free of delays than for it to be one hundred percent reliable. In such situations an **unreliable** service is implemented by omitting the requirement for acknowledgements for the data received. Error checking may be done by the receiver on each block of data, and when one is detected (even when it is only single unknown) the complete data block discarded.
    - When unreliable service is implemented in a given layer, reliability is typically implemented on some higher layer

OPEN SYSTEMS INTERCONNECTION (OSI) MODEL

The Needs of Standard in Network Communication

As we have seen in the previous sections, many software and hardware manufacturers supply products for linking computers in a network. Networking is fundamentally a form of communication, so the need for manufacturers to take steps to ensure that their products could interact became apparent early in the development of networking technology. As networks and suppliers of networking products have spread across the world, the need for standardization has only increased. To address the issues surrounding standardization, several independent organizations have created standard design specifications for computer-networking products. When these standards are adhered to, communication is possible between hardware and software products produced by a variety of vendors.

Network Communications

Network activity involves sending data from one computer to another. This complex process can be broken into discrete, sequential tasks. The sending computer must:

1. Recognize the data.
2. Divide the data into manageable chunks.
3. Add information to each chunk of data to determine the location of the data and to identify the receiver.
4. Add timing and error-checking information.
5. Put the data on the network and send it on its way.

Network client software operates at many different levels within the sending and receiving computers. Each of these levels, or tasks, is governed by one or more protocols. These protocols, or rules of behavior, are standard specifications for formatting and moving the data. When the sending and receiving computers follow the same protocols, communication is assured. Because of this layered structure, this is often referred to as the *protocol stack*.

With the rapid growth of networking hardware and software, a need arose for standard protocols that could allow hardware and software from different vendors to communicate. In response, two primary sets of standards were developed: the OSI reference model and a modification of that standard called Project 802.

Acquiring a clear understanding of these models is an important first step in understanding the technical aspects of how a network functions. Throughout this lesson we refer to various protocols.

The OSI Reference Model

In 1978, the International Organization for Standardization (ISO) released a set of specifications that described network architecture for connecting dissimilar devices. The original document applied to systems that were open to each other because they could all use the same protocols and standards to exchange information.

The OSI reference model is the best-known and most widely used guide for visualizing networking environments. Manufacturers adhere to the OSI reference model when they design network products. It provides a description of how network hardware and software work together in a layered fashion to make communications possible. The model also helps to troubleshoot problems by providing a frame of reference that describes how components are supposed to function.

A Layered Architecture

The OSI reference model architecture divides network communication into seven layers. Each layer covers different network activities, equipment, or protocols. Layering specifies different functions and services as data moves from one computer through the network cabling to another computer. The OSI reference model defines how each layer communicates and works with the layers immediately above and below it. For example, the session layer communicates and works with the presentation and transport layers.

Each layer provides some service or action that prepares the data for delivery over the network to another computer. The lowest layers define the network's physical media and related tasks, such as putting data bits onto the network interface cards (NICs) and cable. The highest layers define how applications access communication services. The higher the layer, the more complex its task.

The layers are separated from each other by boundaries called interfaces. All requests are passed from one layer, through the interface, to the next layer. Each layer builds upon the standards and activities of the layer below it.

Relationships among OSI Reference Model Layers

Each layer provides services to the next-higher layer and shields the upper layer from the details of how the services below it are actually implemented. At the same time, each layer appears to be in direct communication with its associated layer on the other computer. This provides a logical, or virtual, communication between peer layers, as shown in Figure 1.8.1 In reality, actual communication between adjacent layers takes place on one computer only. At each layer, software implements network functions according to a set of protocols.

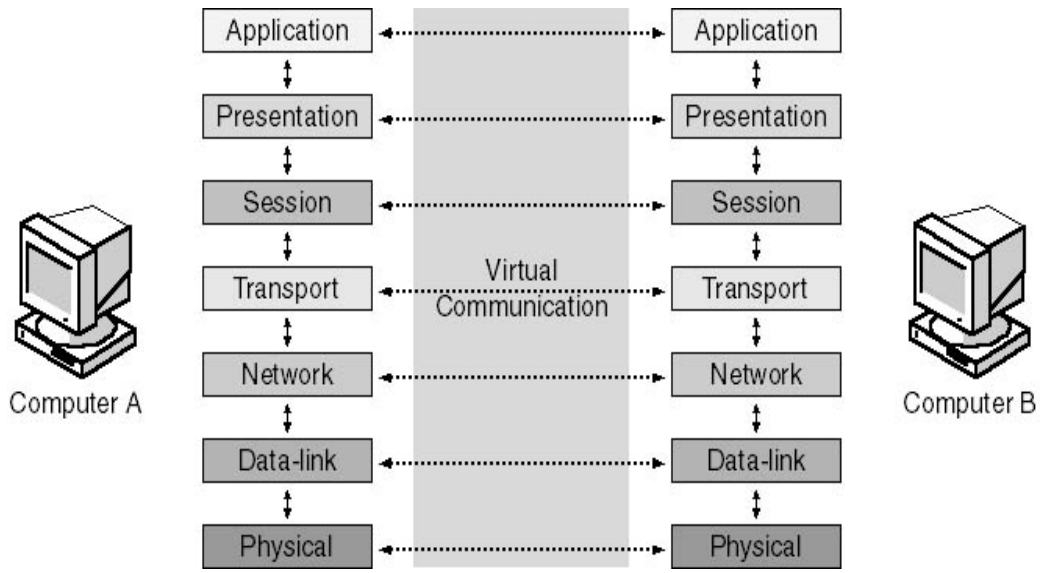


Figure Relationships *among OSI layers*

Before data is passed from one layer to another, it is broken down into packets, or units of information, which are transmitted as a whole from one device to another on a network. The network passes a packet from one software layer to another in the same order as that

of the layers. At each layer, the software adds additional formatting or addressing to the packet, which is needed for the packet to be successfully transmitted across the network.

At the receiving end, the packet passes through the layers in reverse order. A software utility at each layer reads the information on the packet, strips it away, and passes the packet up to the next layer. When the packet is finally passed up to the application layer, the addressing information has been stripped away and the packet is in its original form, which is readable by the receiver.

With the exception of the lowest layer in the OSI networking model, no layer can pass information directly to its counterpart on another computer. Instead, information on the sending computer must be passed down through each successive layer until it reaches the physical layer. The information then moves across the networking cable to the receiving computer and up that computer's networking layers until it arrives at the corresponding layer. For example, when the network layer sends information from computer A, the information moves down through the data-link and physical layers on the sending side, over the cable, and up the physical and data-link layers on the receiving side to its final destination at the network layer on computer B.

In a client/server environment, an example of the kind of information sent from the network layer on computer A to the network layer on computer B would be a network address, with perhaps some error-checking information added to the packet.

Interaction between adjacent layers occurs through an interface. The interface defines the services offered by the lower networking layer to the upper one and further defines how those services will be accessed. In addition, each layer on one computer appears to be communicating directly with the same layer on another computer.

The following sections describe the purpose of each of the seven layers of the OSI reference model, and identify the services that each provides to adjacent layers. Beginning at the

top of the stack (layer 7, the application layer), we work down to the bottom (layer 1, the physical layer).

Application Layer

Layer 7, the topmost layer of the OSI reference model, is the *application layer*. This layer relates to the services that directly support user applications, such as software for file transfers, database access, and e-mail. In other words, it serves as a window through which application processes can access network services. A message to be sent across the network enters the OSI reference model at this point and exits the OSI reference model's application layer on the receiving computer. Application-layer protocols can be programs in themselves, such as File Transfer Protocol (FTP), or they can be used by other programs, such as Simple Mail Transfer Protocol (SMTP), used by most e-mail programs, to redirect data to the network. The lower layers support the tasks that are performed at the application layer. These tasks include general network access, flow control, and error recovery.

Presentation Layer

Layer 6, the *presentation layer*, defines the format used to exchange data among networked computers. Think of it as the network's translator. When computers from dissimilar systems—such as IBM, Apple, and Sun—need to communicate, a certain amount of translation and byte reordering must be done. Within the sending computer, the presentation layer translates data from the format sent down from the application layer into a commonly recognized, intermediary format. At the receiving computer, this layer translates the intermediary format into a format that can be useful to that computer's application layer. The presentation layer is responsible for converting protocols, translating the data, encrypting the data, changing or converting the character set, and expanding graphics commands. The presentation layer also manages data compression to reduce the number of bits that need to be transmitted.

The *redirector*, which redirects input/output (I/O) operations to resources on a server, operates at this layer.

Session Layer

Layer 5, the *session layer*, allows two applications on different computers to open, use, and close a connection called a *session*. (A session is a highly structured dialog between two workstations.) The session layer is responsible for managing this dialog. It performs name-recognition and other functions, such as security, that are needed to allow two applications to communicate over the network.

The session layer synchronizes user tasks by placing checkpoints in the data stream. The checkpoints break the data into smaller groups for error detection. This way, if the network fails, only the data after the last checkpoint has to be retransmitted. This layer also implements dialog control between communicating processes, such as regulating which side transmits, when, and for how long.

Transport Layer

Layer 4, the *transport layer*, provides an additional connection level beneath the session layer. The transport layer ensures that packets are delivered error free, in sequence, and without losses or duplications. At the sending computer, this layer repackages messages, dividing long messages into several packets and collecting small packets together in one package. This process ensures that packets are transmitted efficiently over the network. At the receiving computer, the transport layer opens the packets, reassembles the original messages, and, typically, sends an acknowledgment that the message was received. If a duplicate packet arrives, this layer will recognize the duplicate and discard it.

The transport layer provides flow control and error handling, and participates in solving problems concerned with the transmission and reception of packets. Transmission Control

Protocol (TCP) and Sequenced Packet Exchange (SPX) are examples of transport-layer protocols.

Network Layer

Layer 3, the *network layer*, is responsible for addressing messages and translating logical addresses and names into physical addresses. This layer also determines the route from the source to the destination computer. It determines which path the data should take based on network conditions, priority of service, and other factors. It also manages traffic problems on the network, such as switching and routing of packets and controlling the congestion of data.

If the network adapter on the router cannot transmit a data chunk as large as the source computer sends, the network layer on the router compensates by breaking the data into smaller units. At the destination end, the network layer reassembles the data. Internet Protocol (IP) and Internetwork Packet Exchange (IPX) are examples of network-layer protocols.

Data-Link Layer

Layer 2, the *data-link layer*, sends data frames from the network layer to the physical layer. It controls the electrical impulses that enter and leave the network cable. On the receiving end, the data-link layer packages raw bits from the physical layer into data frames. (A data frame is an organized, logical structure in which data can be placed). The electrical representation of the data (bit patterns, encoding methods, and tokens) is known to this layer only.

Figure 1.8.2 shows a simple data frame. In this example, the sender ID represents the address of the computer that is sending the information; the destination ID represents the address of the computer to which the information is being sent. The control information is used for frame type, routing, and segmentation information. The data is the information

itself. The cyclical redundancy check (CRC) provides error correction and verification information to ensure that the data frame is received correctly.

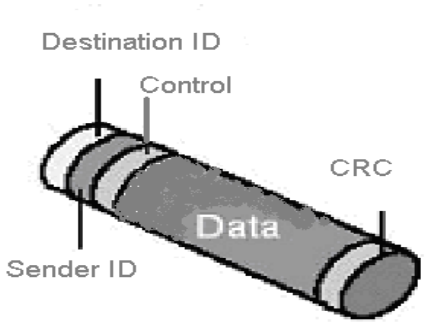


Figure 1.8.2 *A simple data frame*

The data-link layer is responsible for providing error-free transfer of these frames from one computer to another through the physical layer. This allows the network layer to anticipate virtually error-free transmission over the network connection.

Usually, when the data-link layer sends a frame, it waits for an acknowledgment from the recipient. The recipient data-link layer detects any problems with the frame that might have occurred during transmission. Frames that were damaged during transmission or were not acknowledged are then re-sent.

Physical Layer

Layer 1, the bottom layer of the OSI reference model, is the *physical layer*. This layer transmits the unstructured, raw bit stream over a physical medium (such as the network cable). The physical layer is totally hardware-oriented and deals with all aspects of establishing and maintaining a physical link between communicating computers. The physical layer also carries the signals that transmit data generated by each of the higher layers.

This layer defines how the cable is attached to the NIC. For example, it defines how many pins the connector has and the function of each. It also defines which transmission technique will be used to send data over the network cable.

This layer provides data encoding and bit synchronization. The physical layer is responsible for transmitting bits (zeros and ones) from one computer to another, ensuring that when a transmitting host sends a 1 bit, it is received as a 1 bit, not a 0 bit. Because different types of media physically transmit bits (light or electrical signals) differently, the physical layer also defines the duration of each impulse and how each bit is translated into the appropriate electrical or optical impulse for the network cable.

This layer is often referred to as the "hardware layer." Although the rest of the layers can be implemented as firmware (chip-level functions on the NIC), rather than actual software, the other layers are software in relation to this first layer.

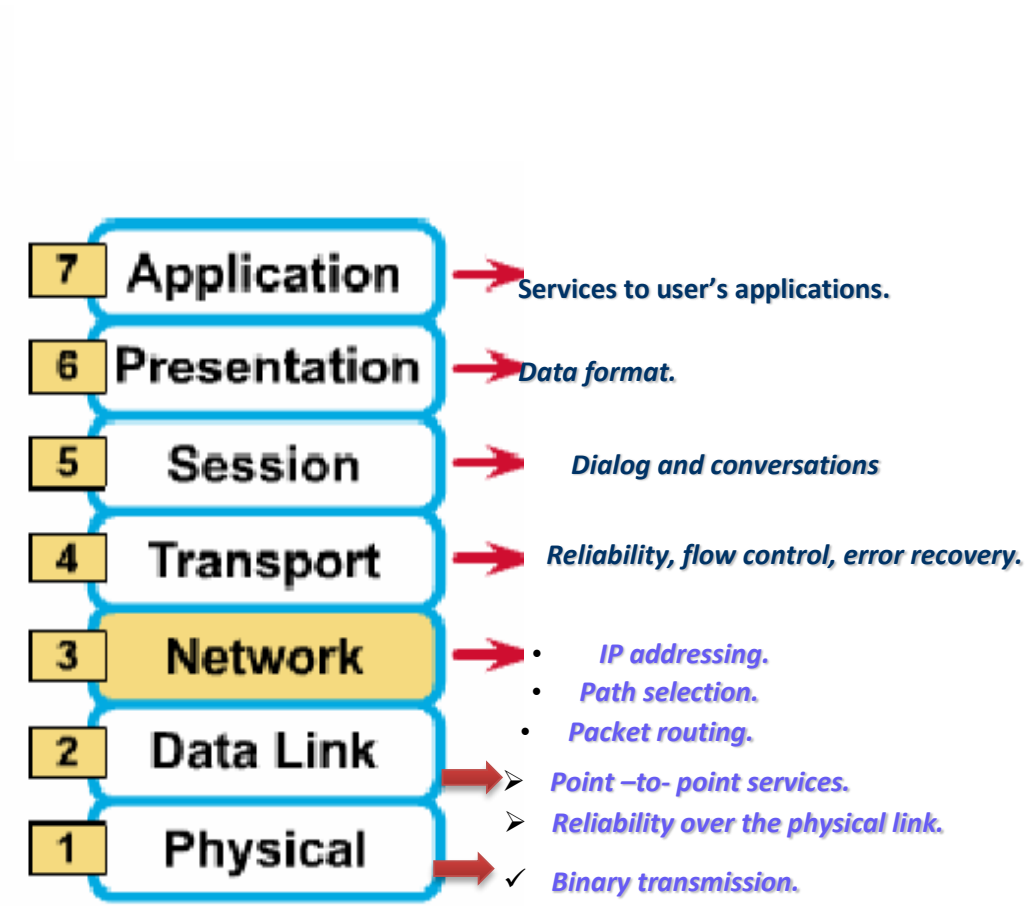
Memorizing the OSI Reference Model

Memorizing the layers of the OSI reference model and their order is very important. Table below provides two ways to help you recall the seven layers of the OSI reference model.

Table *OSI Reference Model Layers*

OSI Layer	Down the Stack	Up the Stack
Application	All	Away
Presentation	People	Pizza
Session	Seem	Sausage
Transport	To	Throw

Network	Need	Not
Data Link	Data	Do
Physical	Processing	Please



Transport Control Protocol / Internet Protocol (TCP/IP)

Transmission Control Protocol/Internet Protocol (TCP/IP) is an industry-standard suite of protocols that provide communications in a heterogeneous (made up of dissimilar elements) environment. In addition, TCP/IP provides a routable, enterprise networking protocol and access to the Internet and its resources. Because of its popularity, TCP/IP has become the de facto standard for what's known as internetworking, the intercommunication in a network that's composed of smaller networks. This lesson examines the TCP/IP protocol and its relationship to the OSI reference model.

TCP/IP has become the standard protocol used for interoperability among many different types of computers. This interoperability is a primary advantage of TCP/IP. Most networks support TCP/IP as a protocol. TCP/IP also supports routing and is commonly used as an internetworking protocol.

Other protocols written specifically for the TCP/IP suite include:

- **SMTP (Simple Mail Transfer Protocol):** E-mail.
- **FTP (File Transfer Protocol):** For exchanging files among computers running TCP/IP.
- **SNMP (Simple Network Management Protocol):** For network management.

Designed to be routable, robust, and functionally efficient, TCP/IP was developed by the United States Department of Defense as a set of wide area network (WAN) protocols. Its purpose was to maintain communication links

between sites in the event of nuclear war. The responsibility for TCP/IP development now resides with the Internet community as a whole. TCP/IP requires significant knowledge and experience on the user's part to install and configure. Using TCP/IP offers several advantages; it:

- **Is an industry standard:** As an industry standard, it is an open protocol. This means it is not controlled by a single company, and is less subject to compatibility issues. It is the de facto/ genuine protocol of the Internet.
- **Contains a set of utilities for connecting dissimilar operating systems:** Connectivity from one computer to another does not depend on the network operating system used on either computer.
- **Uses scalable, cross-platform client-server architecture:** TCP/IP can expand (or shrink) to meet future needs and circumstances. It uses sockets to make the computer operating systems transparent to one another.

TCP/IP is a suite of protocols that provides the foundation for Windows networks and the Internet. The TCP/IP protocol stack is based on a four-layer reference model, including the network interface, internet, transport, and application layers.

The core of TCP/IP services exists at the internet and transport layers. In particular, Address Resolution Protocol (ARP), IP, TCP, User Datagram

Protocol (UDP), and Internet Control Message Protocol (ICMP) are used in all TCP/IP installations.

Exploring the Layers of the TCP/IP Model End-to-end communication through TCP/IP is based on four conceptual steps, or layers.

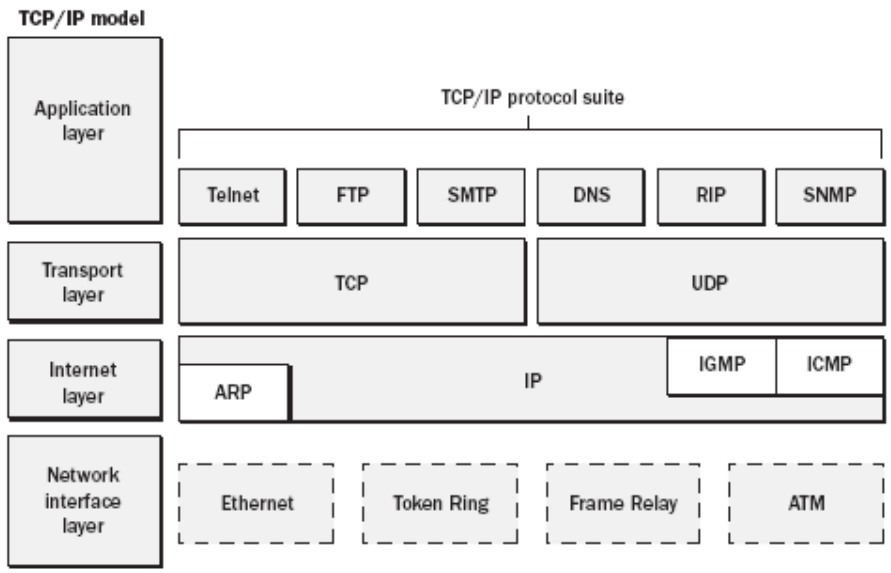


Figure 4-1 Four-layer TCP/IP model and protocol stack

TCP/IP and OSI

The TCP/IP protocol does not exactly match the OSI reference model. Instead of seven layers, it uses only four. Commonly referred to as the Internet Protocol Suite, TCP/IP is broken into the following four layers:

- Network interface layer
- Internet layer
- Transport layer
- Application layer

Each of these layers corresponds to one or more layers of the OSI reference model.

Network Interface Layer

The *network interface layer*, corresponding to the physical and data-link layers of the OSI reference model, communicates directly with the network. It provides the interface between the network architecture (such as token ring, Ethernet) and the Internet layer.

Internet Layer

The *Internet layer*, corresponding to the network layer of the OSI reference model, uses several protocols for routing and delivering packets. Routers are protocol dependent. They function at this layer of the model and are used to forward packets from one network or segment to another. Several protocols work within the Internet layer.

A) Internet Protocol (IP)

Internet Protocol (IP) is a packet-switched protocol that performs addressing and route selection. As a packet is transmitted, this protocol appends a header to the packet so that it can be routed through the network using dynamic routing tables. IP is a connectionless protocol and sends packets without expecting the receiving host to acknowledge receipt. In addition, IP is responsible for packet assembly and disassembly as required by the physical and data-link layers of the OSI reference model. Each IP packet is made up of a source and a destination address, protocol identifier, checksum (a calculated value), and a TTL (which stands for "time to live"). The TTL tells each router on the network between the source and the destination how long the packet has to remain on the network. It works like a countdown counter or clock. As the packet passes through the router, the router deducts the larger of one unit (one second) or the time that the packet was queued for delivery. For example, if a packet has a TTL of 128, it can stay on the network for 128 seconds or 128 hops (each stop, or router, along the way), or any combination of the two.

The purpose of the TTL is to prevent lost or damaged data packets (such as missing e-mail messages) from endlessly wandering the network. When the TTL counts down to zero, the packet is eliminated from the network.

Another method used by the IP to increase the speed of transmission is known as "*ANDing.*" The purpose of ANDing is to determine whether the address is a local or a remote site. If the address is local, IP will ask the Address Resolution Protocol (ARP), discussed in the next section, for the hardware address of the destination machine. If the address is remote, the IP checks its local routing table for a route to the destination. If a route exists, the packet is sent on its way. If no route exists, the packet is sent to the local default gateway and then on its way. [An *AND* is a logical operation that combines the values of two bits (0, 1) or two Boolean values (false, true) that returns a value of 1 (true) if both input values are 1 (true) and returns a 0 (false) otherwise].

B) Address Resolution Protocol (ARP)

Before an IP packet can be forwarded to another host, the hardware address of the receiving machine must be known. The *ARP* determines hardware address (MAC addresses) that corresponds to an IP address. If ARP does not contain the address in its own cache, it broadcasts a request for the address. All hosts on the network process the request and, if they contain a map to that address, pass the address back to the requestor. The packet is then sent on its way, and the new information address is stored in the router's cache.

C) Reverse Address Resolution Protocol (RARP)

A RARP server maintains a database of machine numbers in the form of an ARP table (or cache) which is created by the system administrator. In contrast to ARP, the RARP protocol provides an IP number to a requesting hardware address. When the RARP server receives a request for an IP number from a node on the network, it responds by checking its routing table for the machine number of the requesting node and sending the appropriate IP number back to the requesting node.

D) Internet Control Message Protocol (ICMP)

The *ICMP* is used by IP and higher-level protocols to send and receive status reports about information being transmitted. Routers commonly use ICMP to control the flow, or speed, of data between themselves. If the flow of data is too fast for a router, it requests that other routers slow down.

The two basic categories of ICMP messages are reporting errors and sending queries.

Transport Layer

The *transport layer*, corresponding to the transport layer of the OSI reference model, is responsible for establishing and maintaining end-to-end communication between two hosts. The transport layer provides acknowledgment of receipt, flow control, and sequencing of packets. It also handles retransmissions of packets. The transport layer can use either TCP or User Datagram Protocol (UDP) protocols depending on the requirements of the transmission.

Transmission Control Protocol (TCP)

The TCP is responsible for the reliable transmission of data from one node to another. It is a connection-based protocol and establishes a connection (also known as a session, virtual circuit, or link), between two machines before any data is transferred. To establish a reliable connection, TCP uses what is known as a "three-way handshake." This establishes the port number and beginning sequence numbers from both sides of the transmission. The handshake contains three steps:

1. The requestor sends a packet specifying the port number it plans to use and its initial sequence number (ISN) to the server.

2. The server acknowledges with its ISN, which consists of the requestor's ISN, plus 1.
3. The requestor acknowledges the acknowledgement with the server's ISN, plus 1.

In order to maintain a reliable connection, each packet must contain:

- A source and destination TCP port number.
- A sequence number for messages that must be broken into smaller pieces.
- A checksum to ensure that information is sent without error.
- An acknowledgement number that tells the sending machine which pieces of the message have arrived.
- TCP Sliding Windows.

Ports, Sockets, and Sliding Windows

Protocol port numbers are used to reference the location of a particular application or process on each machine (in the application layer). Just as an IP address identifies the address of a host on the network, the port address identifies the application to the transport layer, thus providing a complete connection for one application on one host to an application on another host. Applications and services (such as file and print services or telnet) can configure up to 65,536 ports. TCP/IP applications and services typically use the first 1023 ports. The Internet Assigned Numbers Authority (IANA) has assigned these as standard, or default, ports. Any client applications dynamically assign port numbers as needed. A port and a node address together make up a socket.

Services and applications use sockets to establish connections with another host. If applications need to guarantee the delivery of data, the socket chooses the connection-oriented service (TCP). If the applications do not need to guarantee data delivery, the socket chooses the connectionless service (UDP).

A sliding window is used by TCP for transferring data between hosts. It regulates how much information can be passed over a TCP connection before the receiving host must send an acknowledgement. Each computer has both a send and a receive window that it utilizes to buffer data and make the communication process more efficient. A sliding window allows the sending computer to transmit data in a stream without having to wait for each packet to be acknowledged. This allows the receiving machine to receive packets out of order and reorganize them while it waits for more packets. The sending window keeps track of data that has been sent, and if an acknowledgement is not received within a given amount of time, the packets are re-sent.

User Datagram Protocol (UDP)

A connectionless protocol, the *UDP*, is responsible for end-to-end transmission of data. Unlike TCP, however, UDP does not establish a connection. It attempts to send the data and to verify that the destination host actually receives the data. UDP is best used to send small amounts of data for which guaranteed delivery is not required. While UDP uses ports, they are different from TCP ports; therefore, they can use the same numbers without interference.

Application Layer

Corresponding to the session, presentation, and application layers of the OSI reference model, the *application layer* connects applications to the network. Two application programming interfaces (APIs) provide access to the TCP/IP transport protocols—Windows Sockets and NetBIOS.

Windows Sockets Interface

Windows Sockets (WinSock) is a networking API designed to facilitate communication among different TCP/IP applications and protocol stacks. It was established so that applications using TCP/IP could write to a standard interface. WinSock is derived from the original sockets that API created for the BSD Unix operating system. WinSock provides a common interface for the applications and protocols that exist near the top of the TCP/IP reference model. Any program or application written using the WinSock API can communicate with any TCP/IP protocol and vice versa

Addressing and Naming

IP addressing

IP addresses are represented by a 32-bit unsigned binary value. It is usually expressed in a dotted decimal format. For example, 9.167.5.8 is a valid IP 66 TCP/IP Tutorial and Technical Overview address. The numeric form is used by IP software. The mapping between the IP address and an easier-to-read symbolic name, for example myhost.ibm.com, is done by the Domain Name System (DNS).

IP addresses are used by the IP protocol to uniquely identify a host on the Internet (or more generally, any internet). Strictly speaking, an IP address identifies an interface that is capable of sending and receiving IP datagrams. One system can have multiple such interfaces. However, both hosts and routers must have at least one IP address, so this simplified definition is acceptable. IP datagrams (the basic data packets exchanged between hosts) are transmitted by a physical network attached to the host. Each IP datagram contains a source IP address and a destination IP address. To send a datagram to a certain IP destination, the target IP address must be translated or mapped to a physical address. This may require transmissions on the network to find out the destination's physical network address. (For example, on LANs, the Address Resolution is used to translate IP addresses to physical MAC addresses.)

IP addressing standards are described in RFC 1166 – Internet Numbers. To identify a host on the Internet, each host is assigned an address, the IP address, or in some cases, the Internet address. When the host is attached to more than one network, it is called multi-homed and has one IP address for each network interface. The IP address consists of a pair of numbers:

```
IP address = <network number><host number>
```

The network number portion of the IP address is administered by one of three

Regional Internet Registries (RIR):

- **American Registry for Internet Numbers (ARIN):** This registry is responsible for the administration and registration of Internet Protocol (IP) numbers for North America, South America, the Caribbean and sub-Saharan Africa.
- **Reseaux IP Europeens (RIPE):** This registry is responsible for the administration and registration of Internet Protocol (IP) numbers for Europe, Middle East, parts of Africa.
- **Asia Pacific Network Information Centre (APNIC):** This registry is responsible for the administration and registration of Internet Protocol (IP) numbers within the Asia Pacific region.

IP addresses are 32-bit numbers represented in a dotted decimal form (as the decimal representation of four 8-bit values concatenated with dots). For example, 128.2.7.9 is an IP address with 128.2 being the network number and 7.9 being the host number. The rules used to divide an IP address into its network and host parts are explained below.

The binary format of the IP address 128.2.7.9 is:

```
10000000 00000010 00000111 00001001
```

Class-based IP addresses

The first bits of the IP address specify how the rest of the address should be separated into its **network** and **host** part. The terms network address and netID are sometimes used instead of network number, but the formal term, used in RFC 1166, is network number. Similarly, the terms host address and hostID are sometimes used instead of host number.

There are five classes of IP addresses. They are shown in Figure 4.2.

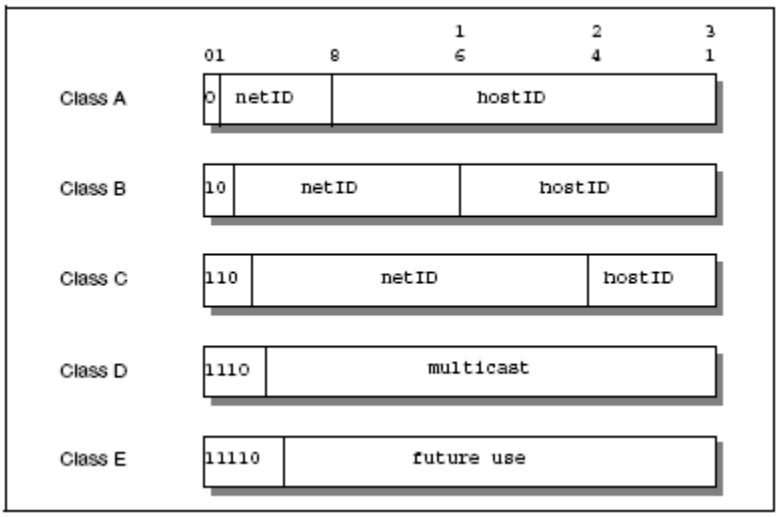


Figure 4.2. IP - Assigned classes of IP addresses

Where:

- **Class A addresses**: These addresses use 7 bits for the <network> and 24 bits for the <host> portion of the IP address. This allows for 2^7-2 (126) networks each with 2^{24}-2 (16777214) hosts; a total of over 2 billion addresses.
- **Class B addresses**: These addresses use 14 bits for the <network> and 16 bits for the <host> portion of the IP address. This allows for 2^{14}-2 (16382) networks each with 2^{16}-2 (65534) hosts; a total of over 1 billion addresses.
- **Class C addresses**: These addresses use 21 bits for the <network> and 8 bits for the <host> portion of the IP address. That allows for 2^{21}-2 (2097150) networks each with 2^8-2 (254) hosts; a total of over half a billion addresses.
- **Class D addresses**: These addresses are reserved for multicasting (a sort of broadcasting, but in a limited area, and only to hosts using the same class D address).
- **Class E addresses**: These addresses are reserved for **future use**.

A Class A address is suitable for networks with an extremely large number of hosts. Class C addresses are suitable for networks with a small number of hosts. This means that medium-sized networks (those with more than 254 hosts or where there is an expectation of more than 254 hosts) must use Class B addresses. However, the number of small- to medium-sized networks has been growing very rapidly. It was feared that if this growth had been allowed to continue unabated, all of the available Class B network addresses would have been used by the mid-1990s. This was termed the IP address exhaustion problem. (The number of networks on the Internet has been approximately doubling annually for a number of years. However, the usage of the Class A, B, and C networks differs greatly. Nearly all of the new networks assigned in the late 1980s were Class B, and in 1990 it became apparent that if this trend continued, the last Class B network number would be assigned during 1994. On the other hand, Class C networks were hardly being used.)

The division of an IP address into two parts also separates the responsibility for selecting the complete IP address. The network number portion of the address is assigned by the RIRs. The host number portion is assigned by the authority controlling the network. As shown in the next section, the host number can be further subdivided: this division is controlled by the authority which manages the network. It is not controlled by the RIRs.

Reserved IP addresses

A component of an IP address with a value all bits 0 or all bits 1 has a special meaning:

- **All bits 0**: An address with all bits zero in the host number portion is interpreted as **this host** (IP address with <host address>=0). All bits zero in the network number portion is **this network** (IP address with <network address>=0). When a host wants to communicate over a network, but does not yet know the network IP address, it may send packets with <network address>=0. Other hosts on the network interpret the address as meaning this network. Their replies contain the fully qualified network address, which the sender records for future use.

- **All bits 1**: An address with all bits one is interpreted as **all networks** or **all hosts**. For example, the following means all hosts on network 128.2 (class B address):

  ```
  128.2.255.255
  ```

  This is called a directed broadcast address because it contains both a valid <network address> and a broadcast <host address>.
- **Loopback**: The class A network 127.0.0.0 is defined as the loopback network. Addresses from that network are assigned to interfaces that process data within the local system. These loopback interfaces do not access a **physical network**.

IP subnets

Due to the explosive growth of the Internet, the principle of assigned IP addresses became too inflexible to allow easy changes to local network configurations. Those changes might occur when:

- A new type of physical network is installed at a location.
- Growth of the number of hosts requires splitting the local network into two or more separate networks.
- Growing distances require splitting a network into smaller networks, with gateways between them.

To avoid having to request additional IP network addresses, the concept of IP subnetting was introduced. The assignment of subnets is done locally. The entire network still appears as one IP network to the outside world.

The host number part of the IP address is subdivided into a second network number and a host number. This second network is termed a subnetwork or subnet. The main network now consists of a number of subnets. The IP address is interpreted as:

```
<network number><subnet number><host number>
```

The combination of subnet number and host number is often termed the local address or the local portion of the IP address. Subnetting is implemented in a way that is transparent to remote networks. A host within a network that has subnets is aware of the subnetting structure. A host in a different network is not. This remote host still regards the local part of the IP address as a host number.

The division of the local part of the IP address into a subnet number and host number is chosen by the local administrator. Any bits in the local portion can be used to form the subnet. The division is done using a 32-bit subnet mask. Bits with a value of zero bits in the subnet mask indicate positions ascribed to the host number. Bits with a value of one indicate positions ascribed to the subnet number. The bit positions in the subnet mask belonging to the original network number are set to ones but are not used (in some platform configurations, this value was actually specified with zeros instead of ones, but either way it is not used). Like IP addresses, subnet masks are usually written in dotted decimal form.

The special treatment of all bits zero and all bits one applies to each of the three parts of a subnetted IP address just as it does to both parts of an IP address that has not been subnetted (see "Reserved IP addresses"). For example, subnetting a Class B network could use one of the following schemes:

- The first octet is the subnet number; the second octet is the host number. This gives 2^8-2 (254) possible subnets, each having up to 2^8-2 (254) hosts. Recall that we subtract two from the possibilities to account for the all ones and all zeros cases. The subnet mask is 255.255.255.0.
- The first 12 bits are used for the subnet number and the last four for the host number. This gives 2^{12}-2 (4094) possible subnets but only 2^4-2 (14) hosts per subnet. The subnet mask is 255.255.255.240.

In this example, there are several other possibilities for assigning the subnet and host portions of the address. The number of subnets and hosts and any future requirements

should be considered before defining this structure. In the last example, the subnetted Class B network has 16 bits to be divided between the subnet number and the host number fields. The network administrator defines either a larger number of subnets each with a small number of hosts, or a smaller number of subnets each with many hosts.

When assigning the subnet part of the local address, the objective is to assign a number of bits to the subnet number and the remainder to the local address. Therefore, it is normal to use a contiguous block of bits at the beginning of the local address part for the subnet number. This makes the addresses more readable. (This is particularly true when the subnet occupies 8 or 16 bits.) With this approach, either of the subnet masks above are "acceptable" masks. Masks such as 255.255.252.252 and 255.255.255.15 are “unacceptable.” In fact, most TCP/IP implementations do not support non-contiguous subnet masks. Their use is universally discouraged.

Subnetting Basics

In Chapter 2, you learned how to define and find the valid host ranges used in a Class A, Class B, and Class C network address by turning the host bits all off and then all on. This is very good, but here’s the catch: You were defining only one network. What happens if you wanted to take one network address and create six networks from it? You would have to do something called *subnetting*, because that’s what allows you to take one larger network and break it into a bunch of smaller networks.
There are loads of reasons in favor of subnetting, including the following benefits:

- **Reduced network traffic** We all appreciate less traffic of any kind. Networks are no different. Without trusty routers, packet traffic could grind the entire network down to a near standstill. With routers, most traffic will stay on the local network; only packets destined for other networks will pass through the router. Routers create broadcast domains. The more broadcast domains you create, the smaller the broadcast domains and the less network traffic on each network segment.
- **Optimized network performance** This is a result of reduced network traffic.
- **Simplified management** It’s easier to identify and isolate network problems in a group of smaller connected networks than within one gigantic network.

- **Facilitated spanning of large geographical distances** Because WAN links are considerably slower and more expensive than LAN links, a single large network that spans long distances can create problems in every area previously listed. Connecting multiple smaller networks makes the system more efficient.

In the following sections, I am going to move to subnetting a network address. This is the good part—ready?

How to Create Subnets

To create subnetworks, you take bits from the host portion of the IP address and reserve them to define the subnet address. This means fewer bits for hosts, so the more subnets, the fewer bits available for defining hosts.
Later in this chapter, you'll learn how to create subnets, starting with Class C addresses. But before you actually implement subnetting, you need to determine your current requirements as well as plan for future conditions.

To create a subnet follow these steps:

1. Determine the number of required network IDs:

   One for each subnet

   One for each wide area network connection

2. Determine the number of required host IDs per subnet:

   One for each TCP/IP host

   One for each router interface

3. Based on the above requirements, create the following:

   One subnet mask for your entire network

   A unique subnet ID for each physical segment

   A range of host IDs for each subnet

Understanding the Powers of 2

Powers of 2 are important to understand and memorize for use with IP subnetting. To review powers of 2, remember that when you see a number

with another number to its upper right (called an exponent), this means you should multiply the number by itself as many times as the upper number specifies. For example, 2^3 is 2 × 2 × 2, which equals 8. Here's a list of powers of 2 that you should commit to memory:

$2^1 = 2$

$2^2 = 4$

$2^3 = 8$

$2^4 = 16$

$2^5 = 32$

$2^6 = 64$

$2^7 = 128$

$2^8 = 256$

$2^9 = 512$

$2^{10} = 1{,}024$

$2^{11} = 2{,}048$

$2^{12} = 4{,}096$

$2^{13} = 8{,}192$

$2^{14} = 16{,}384$

Before you get stressed out about knowing all these exponents, remember that it's helpful to know them, but it's not absolutely necessary. Here's a little trick since you're working with 2s: Each successive power of 2 is double the previous one.

For example, all you have to do to remember the value of 2^9 is to first know that $2^8 = 256$. Why? Because when you double 2 to the eighth power (256),

you get 2^9 (or 512). To determine the value of 2^{10}, simply start at $2^8 = 256$, and then double it twice.

You can go the other way as well. If you needed to know what 2^6 is, for example, you just cut 256 in half two times: once to reach 2^7 and then one more time to reach 2^6.

Subnet Masks

For the subnet address scheme to work, every machine on the network must know which part of the host address will be used as the subnet address. This is accomplished by assigning a *subnet mask* to each machine. A subnet mask is a 32-bit value that allows the recipient of IP packets to distinguish the network ID portion of the IP address from the host ID portion of the IP address.

The network administrator creates a 32-bit subnet mask composed of 1s and 0s. The 1s in the subnet mask represent the positions that refer to the network or subnet addresses.

Not all networks need subnets, meaning they use the default subnet mask. This is basically the same as saying that a network doesn't have a subnet address. Table 3.1 shows the default subnet masks for Classes A, B, and C. These default masks cannot change. In other words, you can't make a Class B subnet mask read 255.0.0.0. If you try, the host will read that address as invalid and usually won't even let you type it in. For a Class A network, you can't change the first byte in a subnet mask; it must read 255.0.0.0 at a minimum. Similarly, you cannot assign 255.255.255.255, as this is all 1s—a broadcast address. A Class B address must start with 255.255.0.0, and a Class C has to start with 255.255.255.0.

TABLE 3.1 Default Subnet Mask

Class	Format	Default Subnet Mask
A	*network.node.node.node*	255.0.0.0
B	*network.network.node.node*	255.255.0.0
C	*network.network.network.node*	255.255.255.0

Classless Inter-Domain Routing (CIDR)

Another term you need to familiarize yourself with is *Classless Inter-Domain Routing (CIDR)*. It's basically the method that ISPs (Internet service providers) use to allocate a number of addresses to a company, a home—a customer. They provide addresses in a certain block size, something I'll be going into in greater detail later in this chapter.

When you receive a block of addresses from an ISP, what you get will look something like this: 192.168.10.32/28. This is telling you what your subnet mask is. The slash notation (/) means how many bits are turned on (1s). Obviously, the maximum could only be /32 because a byte is 8 bits and there are 4 bytes in an IP address: (4 × 8 = 32). But keep in mind that the largest subnet mask available (regardless of the class of address) can only be a /30 because you've got to keep at least 2 bits for host bits.

Take, for example, a Class A default subnet mask, which is 255.0.0.0. This means that the first byte of the subnet mask is all ones (1s), or 11111111. When referring to a slash notation, you need to count all the 1s bits to figure out your mask. The 255.0.0.0 is considered a /8 because it has 8 bits that are 1s—that is, 8 bits that are turned on.

A Class B default mask would be 255.255.0.0, which is a /16 because 16 bits are ones (1s):

11111111.11111111.00000000.00000000.

Table 3.2 has a listing of every available subnet mask and its equivalent CIDR slash notation.

TABLE 3.2 CIDR Values

Subnet Mask	CIDR Value
255.0.0.0	/8
255.128.0.0	/9
255.192.0.0	/10
255.224.0.0	/11
255.240.0.0	/12
255.248.0.0	/13
255.252.0.0	/14
255.254.0.0	/15

255.255.0.0	/16
255.255.128.0	/17
255.255.192.0	/18
255.255.224.0	/19
255.255.240.0	/20
255.255.248.0	/21
255.255.252.0	/22
255.255.254.0	/23
255.255.255.0	/24
255.255.255.128	/25
255.255.255.192	/26
255.255.255.224	/27
255.255.255.240	/28
255.255.255.248	/29
255.255.255.252	/30

The /8 through /15 can only be used with Class A network addresses. /16 through /23 can be used by Class A and B network addresses. /24 through /30 can be used by Class A, B, and C network addresses. This is a big reason why most companies use Class A network addresses.
Since they can use all subnet masks, they get the maximum flexibility in network design.

Subnetting Class C Addresses

There are many different ways to subnet a network. The right way is the way that works best for you. In a Class C address, only 8 bits are available for defining the hosts. Remember that subnet bits start at the left and go to the right, without skipping bits. This means that the only Class C subnet masks can be the following:

Binary Decimal CIDR

00000000 = 0 / 24

10000000 = 128 /25

11000000 = 192 / 26

11100000 = 224 / 27

11110000 = 240 / 28

11111000 = 248 / 29

11111100 = 252 / 30

We can't use a /31 or /32 because we have to have at least 2 host bits for assigning IP addresses to hosts. In the past, I never discussed the /25 in a Class C network. Cisco always had been concerned with having at least 2 subnet bits, but now, because of Cisco recognizing the Ip subnet zero command in its curriculum and exam objectives, we can use just 1 subnet bit.

In the following sections, I'm going to teach you an alternate method of Subnetting that makes it easier to subnet larger numbers in no time. Trust me, you need to be able to subnet fast!

Subnetting a Class C Address: The Fast Way!

When you've chosen a possible subnet mask for your network and need to determine the number of subnets, valid hosts, and broadcast addresses of a subnet that the mask provides, all you need to do is answer five simple questions:

- How many subnets does the chosen subnet mask produce?
- How many valid hosts per subnet are available?
- What are the valid subnets?
- What's the broadcast address of each subnet?
- What are the valid hosts in each subnet?

At this point, it's important that you both understand and have memorized your powers of 2. Please refer to the sidebar "Understanding the Powers of 2" earlier in this chapter if you need some help. Here's how you get the answers to those five big questions:

- ***How many subnets?*** 2^x = number of subnets. *x* is the number of masked bits, or the 1s. For example, in 11000000, the number of 1s gives us 2^2 subnets. In this example, there are 4 subnets.
- ***How many hosts per subnet?*** $2^y - 2$ = number of hosts per subnet. *y* is the number of unmasked bits, or the 0s. For example, in 11000000, the number of 0s gives us $2^6 - 2$ hosts. In this example, there are 62 hosts per subnet. You need to

subtract 2 for the subnet address and the broadcast address, which are not valid hosts.

- ***What are the valid subnets?*** 256 – subnet mask = block size, or increment number. An example would be 256 – 192 = 64. The block size of a 192 mask is always 64. Start counting at zero in blocks of 64 until you reach the subnet mask value and these are your subnets. 0, 64, 128, 192. Easy, huh?
- ***What's the broadcast address for each subnet?*** Now here's the really easy part. Since we counted our subnets in the last section as 0, 64, 128, and 192, the broadcast address is always the number right before the next subnet. For example, the 0 subnet has a broadcast address of 63 because the next subnet is 64. The 64 subnet has a broadcast address of 127 because the next subnet is 128. And so on. And remember, the broadcast address of the last subnet is always 255.
- ***What are the valid hosts?*** Valid hosts are the numbers between the subnets, omitting the all 0s and all 1s. For example, if 64 is the subnet number and 127 is the broadcast address, then 65–126 is the valid host range—it's *always* the numbers between the subnet address and the broadcast address.

I know this can truly seem confusing. But it really isn't as hard as it seems to be at first—just hang in there! Why not try a few and see for yourself?

Subnetting Practice Examples: Class C Addresses

Here's your opportunity to practice Subnetting Class C addresses using the method I just described. Exciting, isn't it! We're going to start with the first Class C subnet mask and work through every subnet that we can using a Class C address. When we're done, I'll show you how easy this is with Class A and B networks too!

Practice Example #1C: 255.255.255.128 (/25)

Since 128 is 10000000 in binary, there is only 1 bit for Subnetting and 7 bits for hosts. We're going to subnet the Class C network address 192.168.10.0.

192.168.10.0 = Network address

255.255.255.128 = Subnet mask Now, let's

answer the big five:

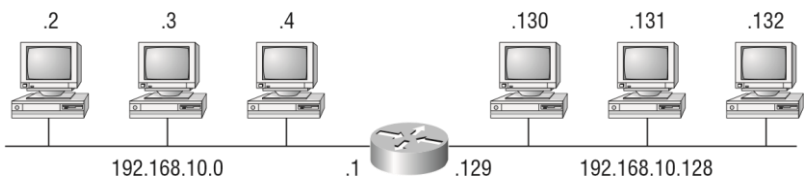
- *How many subnets?* Since 128 is 1 bit on (**1**0000000), the answer would be $2^1 = 2$.
- *How many hosts per subnet?* We have 7 host bits off (1**0000000**), so the equation would be $2^7 – 2 = 126$ hosts.
- *What are the valid subnets?* 256 – 128 = 128. Remember, we'll start at zero and count in our block size, so our subnets are 0, 128.

- ✍ *What's the broadcast address for each subnet?* The number right before the value of the next subnet is all host bits turned on and equals the broadcast address. For the zero subnet, the next subnet is 128, so the broadcast of the 0 subnet is 127.
- ✍ *What are the valid hosts?* These are the numbers between the subnet and broadcast address. The easiest way to find the hosts is to write out the subnet address and the broadcast address. This way, the valid hosts are obvious. The following table shows the 0 and 128 subnets, the valid host ranges of each, and the broadcast address of both subnets :

Subnet	0	128
First host	1	129
Last host	126	254
Broadcast	127	255

Before moving on to the next example, take a look at Figure 3.1. Okay, looking at a Class C /25, it's pretty clear there are two subnets. But so what—why is this significant? Well actually, it's not, but that's not the right question. What you really want to know is what you would do with this information!

FIGURE 3.1 Implementing a Class C /25 logical network



Router#show ip route
[output cut]
C 192.168.10.0 is directly connected to Ethernet 0.
C 192.168.10.128 is directly connected to Ethernet 1.

I know this isn't exactly everyone's favorite pastime, but it's really important, so just hang in there; we're going to talk about Subnetting—period. You need to know that the key to understanding Subnetting is to understand the very reason you need to do it. And I'm going to demonstrate this by going through the process of building a physical network—and let's add a router. (We now have an internetwork, as I truly hope you already know!) All right, because we added that router, in order for the hosts on our internetwork to communicate, they must now have a logical network addressing scheme. We could use IPX or IPv6, but IPv4 is still the most popular, and it also just happens to be what we're studying at the

moment, so that's what we're going with. Okay—now take a look back to Figure 3.1. There are two physical networks, so we're going to implement a logical addressing scheme that allows for two logical networks. As always, it's a really good idea to look ahead and consider likely growth scenarios—both short and long term, but for this example, a /25 will do the trick.

Practice Example #2C: 255.255.255.192 (/26)

In this second example, we're going to subnet the network address 192.168.10.0 using the subnet mask 255.255.255.192.

192.168.10.0 = Network address

255.255.255.192 = Subnet mask Now, let's

answer the big five:

- *How many subnets?* Since 192 is 2 bits on (**11**000000), the answer would be 2^2 = 4 subnets.
- *How many hosts per subnet?* We have 6 host bits off (11**000000**), so the equation would be $2^6 – 2 = 62$ hosts.
- *What are the valid subnets?* 256 – 192 = 64. Remember, we start at zero and count in our block size, so our subnets are 0, 64, 128, and 192.
- *What's the broadcast address for each subnet?* The number right before the value of the next subnet is all host bits turned on and equals the broadcast address. For the zero subnet, the next subnet is 64, so the broadcast address for the zero subnet is 63.
- *What are the valid hosts?* These are the numbers between the subnet and broadcast address.

The easiest way to find the hosts is to write out the subnet address and the broadcast address. This way, the valid hosts are obvious. The following table shows the 0, 64, 128, and 192 subnets, the valid host ranges of each, and the broadcast address of each subnet:

The subnets (do this first)	0	64	128	192
Our first host (perform host addressing last)	1	65	129	193
Our last host	62	126	190	254
The broadcast address (do this second)	63	127	191	255

Okay, again, before getting into the next example, you can see that we can now subnet a /26. And what are you going to do with this fascinating information? Implement it! We'll use Figure 3.2 to practice a /26 network implementation.

The /26 mask provides four sub networks, and we need a subnet for each router interface. With this mask, in this example, we actually have room to add another router interface.

Practice Example #3C: 255.255.255.224 (/27)

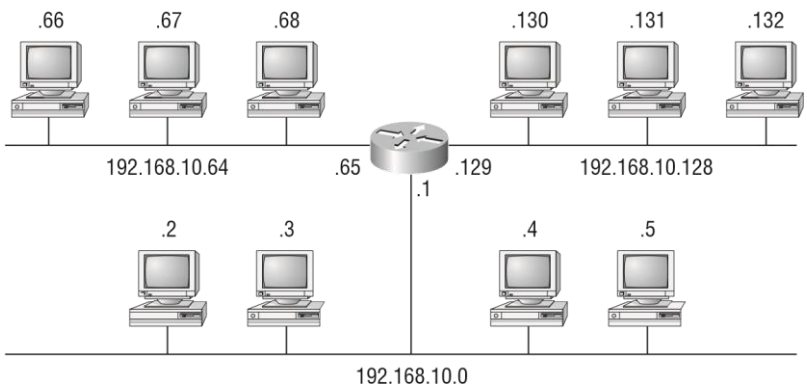
This time, we'll subnet the network address 192.168.10.0 and subnet mask 255.255.255.224.

192.168.10.0 = Network address

255.255.255.224 = Subnet mask

- *How many subnets?* 224 is 11100000, so our equation would be $2^3 = 8$.
- *How many hosts?* $2^5 - 2 = 30$.
- *What are the valid subnets?* 256 – 224 = 32. We just start at zero and count to the subnet mask value in blocks (increments) of 32: 0, 32, 64, 96, 128, 160, 192, and 224.
- *What's the broadcast address for each subnet (always the number right before the next subnet)?*
- *What are the valid hosts (the numbers between the subnet number and the broadcast address)?*

FIGURE 3.2 Implementing a Class C /26 logical network



Router#show ip route
[output cut]
C 192.168.10.0 is directly connected to Ethernet 0
C 192.168.10.64 is directly connected to Ethernet 1
C 192.168.10.128 is directly connected to Ethernet 2

To answer the last two questions, first just write out the subnets, then write out the broadcast addresses—the number right before the next subnet. Last, fill in the host addresses. The following table gives you all the subnets for the 255.255.255.224 Class C subnet mask:

The subnet address	0	32	64	96	128	160	192	224
The first valid host	1	33	65	97	129	161	193	225
The last valid host	30	62	94	126	158	190	222	254
The broadcast address	31	63	95	127	159	191	223	255

Practice Example #4C: 255.255.255.240 (/28) Let's practice on another one:

192.168.10.0 = Network address

255.255.255.240 = Subnet mask

Subnets? 240 is 11110000 in binary. $2^4 = 16$.

Hosts? 4 host bits, or $2^4 – 2 = 14$.

Valid subnets? 256 – 240 = 16. Start at 0: 0 + 16 = 16. 16 + 16 = 32. 32 + 16 = 48. 48 + 16 = 64. 64 + 16 = 80. 80 + 16 = 96. 96 + 16 = 112. 112 + 16 = 128. 128 + 16 = 144. 144 + 16 = 160. 160 + 16 = 176. 176 + 16 = 192. 192 + 16 = 208. 208 + 16 = 224. 224 + 16 = 240.

Broadcast address for each subnet?

Valid hosts?

To answer the last two questions, check out the following table. It gives you the subnets, valid hosts, and broadcast addresses for each subnet. First, find the address of each subnet using the block size (increment). Second, find the broadcast address of each subnet increment (it's always the number right before the next valid subnet), then just fill in the host addresses. The following table shows the available subnets, hosts, and broadcast addresses provided from a Class C 255.255.255.240 mask:

Subnet	0	16	32	48	64	80	96	112	128	144	160	176	192	208	224	240
First host	1	17	33	49	65	81	97	113	129	145	161	177	193	209	225	241
Last host	14	30	46	62	78	94	110	126	142	158	174	190	206	222	238	254

Broadcast 15 31 47 63 79 95 111 127 143 159 175 191 207 223 239 255

Practice Example #5C: 255.255.255.248 (/29) Let's keep practicing:

192.168.10.0 = Network address

255.255.255.248 = Subnet mask

Subnets? 248 in binary = 11111000. $2^5 = 32$.

Hosts? $2^3 - 2 = 6$.

- *Valid subnets?* 256 – 248 = 0, 8, 16, 24, 32, 40, 48, 56, 64, 72, 80, 88, 96, 104, 112, 120, 128 , 136, 144, 152, 160, 168, 176, 184, 192, 200, 208, 216, 224, 232, 240, and 248.

Broadcast address for each subnet?

Valid hosts?

Take a look at the following table. It shows some of the subnets (first four and last four only), valid hosts, and broadcast addresses for the Class C 255.255.255.248 mask:

Subnet	0	8	16	24	…	224	232	240	248
First host	1	9	17	25	…	225	233	241	249
Last host	6	14	22	30	…	230	238	246	254
Broadcast	7	15	23	31	…	231	239	247	255

Practice Example #6C: 255.255.255.252 (/30) Just one more:

192.168.10.0 = Network address

255.255.255.252 = Subnet mask *Subnets?* 64.

Hosts? 2.

- *Valid subnets?* 0, 4, 8, 12, etc., all the way to 252.
- *Broadcast address for each subnet (always the number right before the next subnet)?*
- *Valid hosts (the numbers between the subnet number and the broadcast address)?*

The following table shows you the subnet, valid host, and broadcast address of the first four and last four subnets in the 255.255.255.252 Class C subnet:

Subnetting in Your Head: Class C Addresses

It really is possible to subnet in your head. Even if you don't believe me, I'll show you how. And it's not all that hard either—take the following example:

192.168.10.33 = Node address

255.255.255.224 = Subnet mask

First, determine the subnet and broadcast address of the above IP address. You can do this by answering question 3 of the big five questions: 256 – 224 = 32. 0, 32, 64. The address of 33 falls between the two subnets of 32 and 64 and must be part of the 192.168.10.32 subnet. The next subnet is 64, so the broadcast address of the 32 subnet is 63. (Remember that the broadcast address of a subnet is always the number right before the next subnet.) The valid host range is 33–62 (the numbers between the subnet and broadcast address). This is too easy! Okay, let's try another one. We'll subnet another Class C address:

192.168.10.33 = Node address

255.255.255.240 = Subnet mask

What subnet and broadcast address is the above IP address a member of? 256 – 240 = 16. 0, 16, 32, 48. Bingo—the host address is between the 32 and 48 subnets. The subnet is 192.168.10.32, and the broadcast address is 47 (the next subnet is 48). The valid host range is 33–46 (the numbers between the subnet number and the broadcast address).

Okay, we need to do more, just to make sure you have this down.

You have a node address of 192.168.10.174 with a mask of 255.255.255.240. What is the valid host range?

The mask is 240, so we'd do a 256 – 240 = 16. This is our block size. Just keep adding 16 until we pass the host address of 174, starting at zero, of course: 0, 16, 32, 48, 64, 80, 96, 112, 128, 144, 160, 176. The host address of 174 is between 160 and 176, so the subnet is 160. The broadcast address is 175; the valid host range is 161–174. That was a tough one. One more—just for fun. This is the easiest one of all Class C Subnetting:

192.168.10.17 = Node address

255.255.255.252 = Subnet mask

What subnet and broadcast address is the above IP address a member of? 256 – 252 = 0 (always start at zero unless told otherwise), 4, 8, 12, 16, 20, etc. You've got it! The host address is between the 16 and 20 subnets. The subnet is 192.168.10.16, and the broadcast address is 19. The valid host range is 17–18.

Now that you're all over Class C Subnetting, let's move on to Class B Subnetting. But before we do, let's have a quick review.

What Do We Know?

Okay—here's where you can really apply what you've learned so far, and begin committing it all to memory. This is a very cool section that I've been using in my classes for years. It will really help you nail down Subnetting!

When you see a subnet mask or slash notation (CIDR), you should know the following:

/25 What do we know about a /25?

128 mask

1 bits on and 7 bits off (10000000)

Block size of 128

2 subnets, each with 126 hosts

/26 What do we know about a /26?

192 mask

2 bits on and 6 bits off (11000000)

Block size of 64

4 subnets, each with 62 hosts

/27 What do we know about a /27?

224 mask

3 bits on and 5 bits off (11100000)

Block size of 32

8 subnets, each with 30 hosts

/28 What do we know about a /28?

240 mask

4 bits on and 4 bits off

Block size of 16

16 subnets, each with 14 hosts **/29**

What do we know about a /29?

248 mask

5 bits on and 3 bits off

Block size of 8

32 subnets, each with 6 hosts

/30 What do we know about a /30?

252 mask

6 bits on and 2 bits off

Block size of 4

64 subnets, each with 2 hosts

Regardless of whether you have a Class A, Class B, or Class C address, the /30 mask will provide you with only two hosts, ever. This mask is suited almost exclusively—as well as suggested by Cisco—for use on point-to-point links.

If you can memorize this "What Do We Know?" section, you'll be much better off in your day-to-day job and in your studies. Try saying it out loud, which helps you memorize things— yes, your significant other and/or coworkers will think you've lost it, but they probably already do if you are in the networking field. And if you're not yet in the networking field but are studying all this to break into it, you might as well have people start thinking you're an odd bird now since they will eventually anyway.

It's also helpful to write these on some type of flashcards and have people test your skill. You'd be amazed at how fast you can get Subnetting down if you memorize block sizes as well as this "What Do We Know?" section.

Subnetting Class B Addresses

Before we dive into this, let's look at all the possible Class B subnet masks first. Notice that we have a lot more possible subnet masks than we do with a Class C network address:

255.255.0.0	(/16)		
255.255.128.0	(/17)	255.255.255.0	(/24)
255.255.192.0	(/18)	255.255.255.128	(/25)
255.255.224.0	(/19)	255.255.255.192	(/26)
255.255.240.0	(/20)	255.255.255.224	(/27)
255.255.248.0	(/21)	255.255.255.240	(/28)
255.255.252.0	(/22)	255.255.255.248	(/29)
255.255.254.0	(/23)	255.255.255.252	(/30)

We know the Class B network address has 16 bits available for host addressing. This means we can use up to 14 bits for Subnetting (because we have to leave at least 2 bits for host addressing). Using a /16 means you are not Subnetting with class B, but it is a mask you can use.

By the way, do you notice anything interesting about that list of subnet values—a pattern, maybe? Ah ha! That's exactly why I had you memorize the binary-to-decimal numbers at the beginning of this section. Since subnet mask bits start on the left and move to the right and bits can't be skipped, the numbers are always the same regardless of the class of address. Memorize this pattern.

The process of Subnetting a Class B network is pretty much the same as it is for a Class C, except that you just have more host bits and you start in the third octet.

Use the same subnet numbers for the third octet with Class B that you used for the fourth octet with Class C, but add a zero to the network portion and a 255 to the broadcast section in the fourth octet. The following table shows you an example host range of two subnets used in a Class B 240 (/20) subnet mask:

First subnet	16.0	32.0
Second subnet	31.255	47.255

Just add the valid hosts between the numbers, and you're set!

Mezgebe M. [UU]

Subnetting Practice Examples: Class B Addresses

This section will give you an opportunity to practice Subnetting Class B addresses. Again, I have to mention that this is the same as Subnetting with Class C, except we start in the third octet—with the exact same numbers!

Practice Example #1B: 255.255.128.0 (/17)

172.16.0.0 = Network address

255.255.128.0 = Subnet mask

- *Subnets?* $2^1 = 2$ (same as Class C).
- *Hosts?* $2^{15} – 2 = 32{,}766$ (7 bits in the third octet, and 8 in the fourth).
- *Valid subnets?* 256 – 128 = 128. 0, 128. Remember that Subnetting is performed in the third octet, so the subnet numbers are really 0.0 and 128.0, as shown in the next table. These are the exact numbers we used with Class C; we use them in the third octet and add a 0 in the fourth octet for the network address.
- *Broadcast address for each subnet?*
- *Valid hosts?*

The following table shows the two subnets available, the valid host range, and the broadcast address of each:

Subnet	0.0	128.0
First host	0.1	128.1
Last host	127.254	255.254
Broadcast	127.255	255.255

Okay, notice that we just added the fourth octet's lowest and highest values and came up with the answers. And again, it's done exactly the same way as for a Class C subnet. We just use the same numbers in the third octet and added 0 and 255 in the fourth octet—pretty simple huh! I really can't say this enough: It's just not hard; the numbers never change; we just use them in different octets!

Practice Example #2B: 255.255.192.0 (/18)

172.16.0.0 = Network address

255.255.192.0 = Subnet mask

- *Subnets?* $2^2 = 4$.
- *Hosts?* $2^{14} – 2 = 16{,}382$ (6 bits in the third octet, and 8 in the fourth).

- *Valid subnets?* 256 – 192 = 64. 0, 64, 128, 192. Remember that the Subnetting is performed in the third octet, so the subnet numbers are really 0.0, 64.0, 128.0, and 192.0, as shown in the next table.
- *Broadcast address for each subnet?*
- *Valid hosts?*

The following table shows the four subnets available, the valid host range, and the broadcast address of each:

Subnet	0.0	64.0	128.0	192.0
First host	0.1	64.1	128.1	192.1
Last host	63.254	127.254	191.254	255.254
Broadcast	63.255	127.255	191.255	255.255

Again, it's pretty much the same as it is for a Class C subnet—we just added 0 and 255 in the fourth octet for each subnet in the third octet.

Practice Example #3B: 255.255.240.0 (/20)

172.16.0.0 = Network address

255.255.240.0 = Subnet mask

- *Subnets?* $2^4 = 16$.
- *Hosts?* $2^{12} – 2 = 4094$.
- *Valid subnets?* 256 – 240 = 0, 16, 32, 48, etc., up to 240. Notice that these are the same numbers as a Class C 240 mask – we just put them in the third octet and add a 0 and 255 in the fourth octet.
- *Broadcast address for each subnet?*
- *Valid hosts?*

The following table shows the first four subnets, valid hosts, and broadcast addresses in a Class B 255.255.240.0 mask:

Subnet	0.0	16.0	32.0	48.0
First host	0.1	16.1	32.1	48.1
Last host	15.254	31.254	47.254	63.254
Broadcast	15.255	31.255	47.255	63.255

Practice Example #4B: 255.255.254.0 (/23)

172.16.0.0 = Network address

255.255.254.0 = Subnet mask

- *Subnets?* $2^7 = 128$.
- *Hosts?* $2^9 - 2 = 510$.
- *Valid subnets?* 256 – 254 = 0, 2, 4, 6, 8, etc., up to 254.
- *Broadcast address for each subnet?*
- *Valid hosts?*

The following table shows the first five subnets, valid hosts, and broadcast addresses in a Class B 255.255.254.0 mask:

Subnet	0.0	2.0	4.0	6.0	8.0
First host	0.1	2.1	4.1	6.1	8.1
Last host	1.254	3.254	5.254	7.254	9.254
Broadcast	1.255	3.255	5.255	7.255	9.255

Practice Example #5B: 255.255.255.0 (/24)

Contrary to popular belief, 255.255.255.0 used with a Class B network address is not called a Class B network with a Class C subnet mask. It's amazing how many people see this mask used in a Class B network and think it's a Class C subnet mask. This is a Class B subnet mask with 8 bits of Subnetting—it's considerably different from a Class C mask. Subnetting this address is fairly simple:

172.16.0.0 = Network address

255.255.255.0 = Subnet mask

- *Subnets?* $2^8 = 256$.
- *Hosts?* $2^8 - 2 = 254$.
- *Valid subnets?* 256 – 255 = 1. 0, 1, 2, 3, etc., all the way to 255.
- *Broadcast address for each subnet?*
- *Valid hosts?*

The following table shows the first four and last two subnets, the valid hosts, and the broadcast addresses in a Class B 255.255.255.0 mask:

Subnet	0.0	1.0	2.0	3.0	...	254.0	255.0
First host	0.1	1.1	2.1	3.1	...	254.1	255.1
Last host	0.254	1.254	2.254	3.254	...	254.254	255.254
Broadcast	0.255	1.255	2.255	3.255	...	254.255	255.255

Practice Example #6B: 255.255.255.128 (/25)

This is one of the hardest subnet masks you can play with. And worse, it actually is a really good subnet to use in production because it creates over 500 subnets with 126 hosts for each subnet—a nice mixture. So, don't skip over it!

172.16.0.0 = Network address

255.255.255.128 = Subnet mask

- *Subnets?* $2^9 = 512$.
- *Hosts?* $2^7 - 2 = 126$.
- *Valid subnets?* Okay, now for the tricky part. 256 – 255 = 1. 0, 1, 2, 3, etc. for the third octet. But you can't forget the one subnet bit used in the fourth octet. Remember when I showed you how to figure one subnet bit with a Class C mask? You figure this the same way. (Now you know why I showed you the 1-bit subnet mask in the Class C section— to make this part easier.) You actually get two subnets for each third octet value, hence the 512 subnets. For example, if the third octet is showing subnet 3, the two subnets would actually be 3.0 and 3.128.
- *Broadcast address for each subnet?*
- *Valid hosts?*

The following table shows how you can create subnets, valid hosts, and broadcast addresses using the Class B 255.255.255.128 subnet mask (the first eight subnets are shown, and then the last two subnets):

Subnet	0.0	0.128	1.0	1.128	2.0	2.128	3.0	3.128	...	255.0	255.128
First host	0.1	0.129	1.1	1.129	2.1	2.129	3.1	3.129	...	255.1	255.129
Last host	0.126	0.254	1.126	1.254	2.126	2.254	3.126	3.254	...	255.126	255.254
Broadcast	0.127	0.255	1.127	1.255	2.127	2.255	3.127	3.255	...	255.127	255.255

Practice Example #7B: 255.255.255.192 (/26)

Now, this is where Class B Subnetting gets easy. Since the third octet has a 255 in the mask section, whatever number is listed in the third octet is a subnet number. However, now that we have a subnet number in the fourth octet, we can subnet this octet just as we did with Class C Subnetting. Let's try it out:

172.16.0.0 = Network address

255.255.255.192 = Subnet mask

Subnets? $2^{10} = 1024$.

Hosts? $2^6 - 2 = 62$.

Valid subnets? 256 – 192 = 64. The subnets are shown in the following table. Do these numbers look familiar?

Broadcast address for each subnet?

Valid hosts?

The following table shows the first eight subnet ranges, valid hosts, and broadcast addresses:

Subnet	0.0	0.64	0.128	0.192	1.0	1.64	1.128	1.192
First host	0.1	0.65	0.129	0.193	1.1	1.65	1.129	1.193
Last host	0.62	0.126	0.190	0.254	1.62	1.126	1.190	1.254
Broadcast	0.63	0.127	0.191	0.255	1.63	1.127	1.191	1.255

Notice that for each subnet value in the third octet, you get subnets 0, 64, 128, and 192 in the fourth octet.

Subnetting in Your Head: Class B Addresses

Are you nuts? Subnet Class B addresses in our heads? It's actually easier than writing it out— I'm not kidding! Let me show you how:

Question: What subnet and broadcast address is the IP address 172.16.10.33 255.255.255.224 (/27) a member of?

Answer: The interesting octet is the fourth octet. 256 – 224 = 32. 32 + 32 = 64. Bingo: 33 is between 32 and 64. However, remember that the third octet is considered part of the subnet, so the answer would be the 10.32 subnet. The broadcast is 10.63, since 10.64 is the next subnet. That was a pretty easy one.

Question: What subnet and broadcast address is the IP address 172.16.66.10 255.255.192.0 (/18) a member of?

Answer: The interesting octet is the third octet instead of the fourth octet. 256 – 192 = 64. 0, 64, 128. The subnet is 172.16.64.0. The broadcast must be 172.16.127.255 since 128.0 is the next subnet.

Question: What subnet and broadcast address is the IP address 172.16.50.10 255.255.224.0 (/19) a member of?

Answer: 256 – 224 = 0, 32, 64 (remember, we always start counting at zero (0)). The subnet is 172.16.32.0, and the broadcast must be 172.16.63.25 since 64.0 is the next subnet.

Question: What subnet and broadcast address is the IP address 172.16.46.255 255.255.240.0 (/20) a member of?

Answer: 256 – 240 = 16. The third octet is interesting to us. 0, 16, 32, 48. This subnet address must be in the 172.16.32.0 subnet, and the broadcast must be 172.16.47.255 since 48.0 is the next subnet. So, yes, 172.16.46.255 is a valid host.

Question: What subnet and broadcast address is the IP address 172.16.45.14 255.255.255.252 (/30) a member of?

Answer: Where is the interesting octet? 256 – 252 = 0, 4, 8, 12, 16 (in the fourth octet). The subnet is 172.16.45.12, with a broadcast of 172.16.45.15 because the next subnet is 172.16.45.16.

Question: What is the subnet and broadcast address of the host 172.16.88.255/20?

Answer: What is a /20? If you can't answer this, you can't answer this question, can you? A /20 is 255.255.240.0, which gives us a block size of 16 in the third octet, and since no subnet bits are on in the fourth octet, the answer is always 0 and 255 in the fourth octet. 0, 16, 32, 48, 64, 80, 96…bingo. 88 is between 80 and 96, so the subnet is 80.0 and the broadcast address is 95.255.

Question: A router receives a packet on an interface with a destination address of 172.16.46.191/26 . What will the router do with this packet?

Answer: Discard it. Do you know why? 172.16.46.191/26 is a 255.255.255.192 mask, which gives us a block size of 64. Our subnets are then 0, 64, 128, 192. 191 is the broadcast address of the 128 subnet, so a router, by default, will discard any broadcast packets.

Subnetting Class A Addresses

Class A Subnetting is not performed any differently than Classes B and C, but there are 24 bits to play with instead of the 16 in a Class B address and the 8 in a Class C address. Let's start by listing all the Class A masks:

255.0.0.0 (/ 8)	
255.128.0.0 (/9)	255.255.240.0 (/20)
255.192.0.0 (/10)	255.255.248.0 (/21)
255.224.0.0 (/11)	255.255.252.0 (/22)
255.240.0.0 (/12)	255.255.254.0 (/23)
255.248.0.0 (/13)	255.255.255.0 (/24)

255.252.0.0 (/14)	255.255.255.128 (/25)
255.254.0.0 (/15)	255.255.255.192 (/26)
255.255.0.0 (/16)	255.255.255.224 (/27)
255.255.128.0 (/17)	255.255.255.240 (/28)
255.255.192.0 (/18)	255.255.255.248 (/29)
255.255.224.0 (/19)	255.255.255.252 (/30)

That's it. You must leave at least 2 bits for defining hosts. And I hope you can see the pattern by now. Remember, we're going to do this the same way as a Class B or C subnet. It's just that, again, we simply have more host bits and we just use the same subnet numbers we used with Class B and C, but we start using these numbers in the second octet.

Subnetting Practice Examples: Class A Addresses

When you look at an IP address and a subnet mask, you must be able to distinguish the bits used for subnets from the bits used for determining hosts. This is imperative. If you're still struggling with this concept, please reread the section "IP Addressing" in Chapter 2. It shows you how to determine the difference between the subnet and host bits and should help clear things up.

Practice Example #1A: 255.255.0.0 (/16)

Class A addresses use a default mask of 255.0.0.0, which leaves 22 bits for Subnetting since you must leave 2 bits for host addressing. The 255.255.0.0 mask with a Class A address is using 8 subnet bits.

Subnets? $2^8 = 256$.

Hosts? $2^{16} - 2 = 65{,}534$.

Valid subnets? What is the interesting octet? 256 – 255 = 1. 0, 1, 2, 3, etc. (all in the second octet). The subnets would be 10.0.0.0, 10.1.0.0, 10.2.0.0, 10.3.0.0, etc., up to 10.255.0.0.

Broadcast address for each subnet?

Valid hosts?

The following table shows the first two and last two subnets, valid host range, and broadcast addresses for the private Class A 10.0.0.0 network:

Subnet	10.0.0.0	10.1.0.0	…	10.254.0.0	10.255.0.0
First host	10.0.0.1	10.1.0.1	…	10.254.0.1	10.255.0.1

Last host	10.0.255.254	10.1.255.254	…	10.254.255.254 10.255.255.254
Broadcast	10.0.255.255	10.1.255.255	…	10.254.255.255 10.255.255.255

Practice Example #2A: 255.255.240.0 (/20)

255.255.240.0 gives us 12 bits of Subnetting and leaves us 12 bits for host addressing.

Subnets? $2^{12} = 4096$.

Hosts? $2^{12} - 2 = 4094$.

Valid subnets? What is your interesting octet? 256 – 240 = 16. The subnets in the second octet are a block size of 1 and the subnets in the third octet are 0, 16, 32, etc.

Broadcast address for each subnet?

Valid hosts?

The following table shows some examples of the host ranges—the first three and the last subnets:

Subnet	10.0.0.0	10.0.16.0	10.0.32.0	…	10.255.240.0
First host	10.0.0.1	10.0.16.1	10.0.32.1	…	10.255.240.1
Last host	10.0.15.254	10.0.31.254	10.0.47.254	…	10.255.255.254
Broadcast	10.0.15.255	10.0.31.255	10.0.47.255	…	10.255.255.255

Practice Example #3A: 255.255.255.192 (/26)

Let's do one more example using the second, third, and fourth octets for Subnetting.

Subnets? $2^{18} = 262{,}144$.

Hosts? $2^{6} - 2 = 62$.

Valid subnets? In the second and third octet, the block size is 1, and in the fourth octet, the block size is 64.

Broadcast address for each subnet?

Valid hosts?

The following table shows the first four subnets and their valid hosts and broadcast addresses in the Class A 255.255.255.192 mask:

Subnet	10.0.0.0	10.0.0.64	10.0.0.128	10.0.0.192
First host	10.0.0.1	10.0.0.65	10.0.0.129	10.0.0.193
Last host	10.0.0.62	10.0.0.126	10.0.0.190	10.0.0.254
Broadcast	10.0.0.63	10.0.0.127	10.0.0.191	10.0.0.255

The following table shows the last four subnets and their valid hosts and broadcast addresses:

Subnet	10.255.255.0	10.255.255.64	10.255.255.128	10.255.255.192
First host	10.255.255.1	10.255.255.65	10.255.255.129	10.255.255.193
Last host	10.255.255.62	10.255.255.126	10.255.255.190	10.255.255.254
Broadcast	10.255.255.63	10.255.255.127	10.255.255.191	10.255.255.255

Subnetting in Your Head: Class A Addresses

This sounds hard, but as with Class C and Class B, the numbers are the same; we just start in the second octet. What makes this easy? You only need to worry about the octet that has the largest block size (typically called the interesting octet; one that is something other than 0 or 255)—for example, 255.255.240.0 (/20) with a Class A network. The second octet has a block size of 1, so any number listed in that octet is a subnet. The third octet is a 240 mask, which means we have a block size of 16 in the third octet. If your host ID is 10.20.80.30, what is your subnet, broadcast address, and valid host range?

The subnet in the second octet is 20 with a block size of 1, but the third octet is in block sizes of 16, so we'll just count them out: 0, 16, 32, 48, 64, 80, 96…voilà! (By the way, you can count by 16s by now, right?) This makes our subnet 10.20.80.0, with a broadcast of 10.20.95.255 because the next subnet is 10.20.96.0. The valid host range is 10.20.80.1 through 10.20.95.254. And yes, no lie! You really can do this in your head if you just get your block sizes nailed!

Okay, let's practice on one more, just for fun!

Host IP: 10.1.3.65/23

First, you can't answer this question if you don't know what a /23, is. It's 255.255.254.0. The interesting octet here is the third one: 256 – 254 = 2. Our subnets in the third octet are 0, 2, 4, 6, etc. The host in this question is in subnet 2.0, and the next subnet is 4.0, so that makes the broadcast address 3.255. And any address between 10.1.2.1 and 10.1.3.254 is considered a valid host.